



Алгоритмы обработки больших данных

L02. Stream intro, flink intro

Апрель, 2026

Смирнов Сергей

smirnov.work@mail.ru

[@serg_v_smirnov](#)

Agenda

1. BigData intro
- 2. *Stream & flink intro***
3. Batch mode, pyflink runtime architecture
4. DataStream operators API, keyed state
5. Stateful stream processing. Windows, Triggers, ProcessFunction, State, Timers, Watermarks
6. State and Chekpoint, State Backends. Table API и SQL API
7. CEP, CDC, ML, Metrics, Failure recovery, Execution configuration, Async IO, Delivery guarantees. Best (worst) practices
8. Flink-SQL, базовый синтаксис, работа с окнами
9. Flink-SQL, соединение потоков, udf, оптимизация
10. Whats new, trends (fluss, streamhouse)

Homework



kafka + python

- *Развернуть кластер kafka*
- *Написать простое приложение по чтению и записи в kafka*

Terminology



Стриминг – это обработка неограниченного потока данных



Батч – работа с ограниченным набором данных



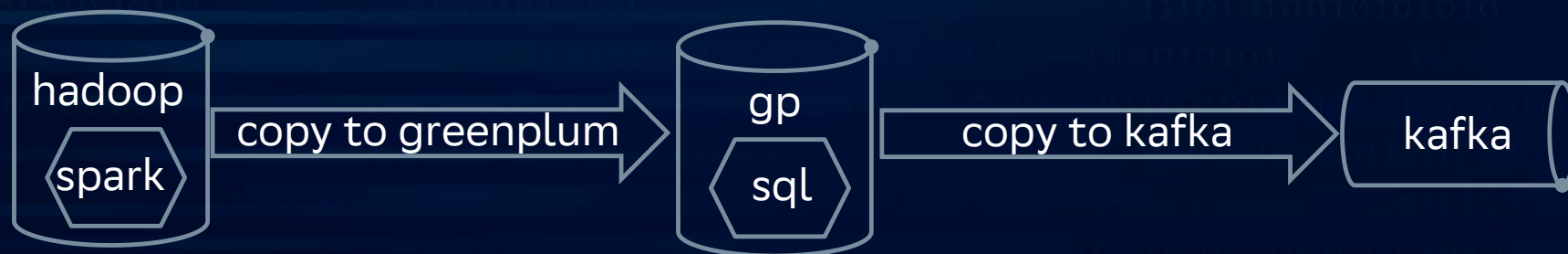
Interim conclusions

Батчинг – это частный случай стриминга

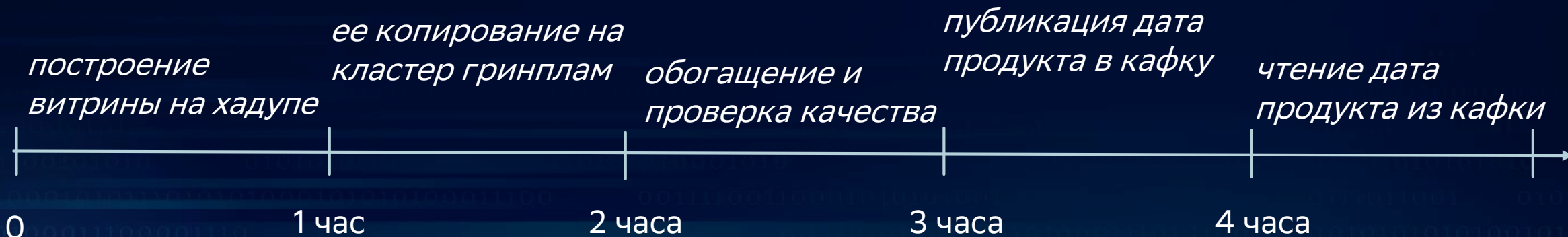
Стриминговый процесс (потокковое решение) работает постоянно (в отличие от батчевых периодических загрузок)

Ничего про сохранение порядка

batch vs micro batch vs streaming 1/3



1. Очистка данных и их обогащение, hadoop
2. Копирование данных на greenplum
3. Обработка, качество данных, обогащение
4. Копирование финальной витрины в кафку
5. Чтение данных из кафки

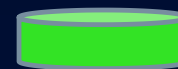


batch vs micro batch vs streaming 2/3



Разделим на 4 части, которые можно независимо обрабатывать

Обработка каждой части для каждого шага составляет 20 минут
(больше 1/3 часа, налог на разделение)



Работа завершена

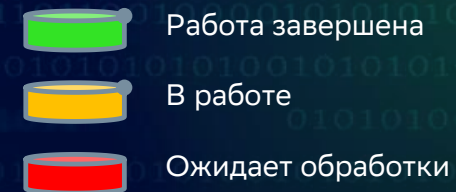


В работе



Ожидает обработки

batch vs micro batch vs streaming 3/3



$t \in (0; 20)$



$t \in (20; 40)$



$t \in (40; 60)$



$t \in (60; 80)$



$t \in (80; 100)$



$t \in (100; 120)$



...

$t \in (140; 160)$



$t \in (160; 180)$



batch vs micro batch vs streaming, conclusions

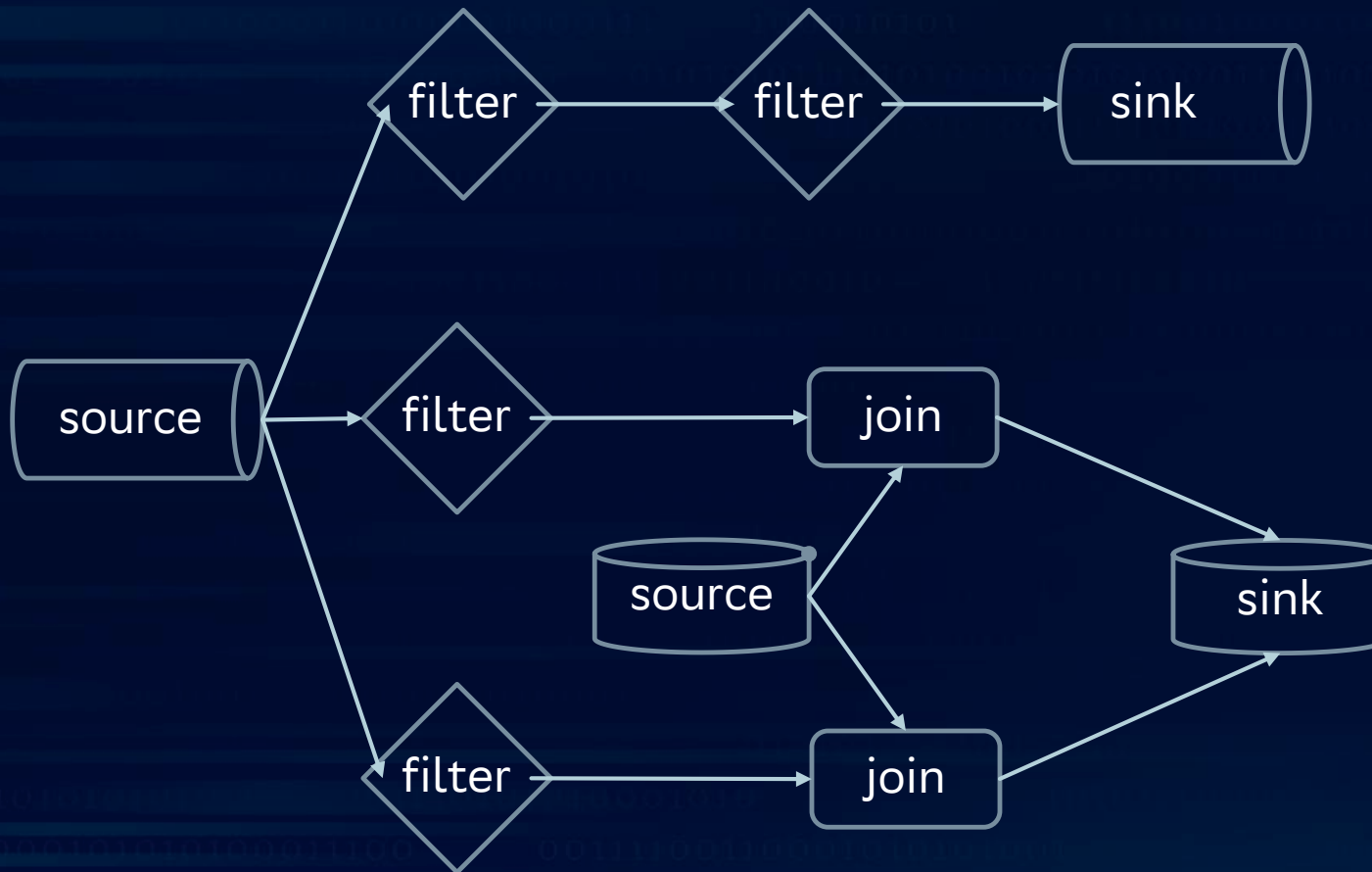
Батч:	300 минут
Микро батч:	160 минут
Стриминг:	<100 минут

Simple example

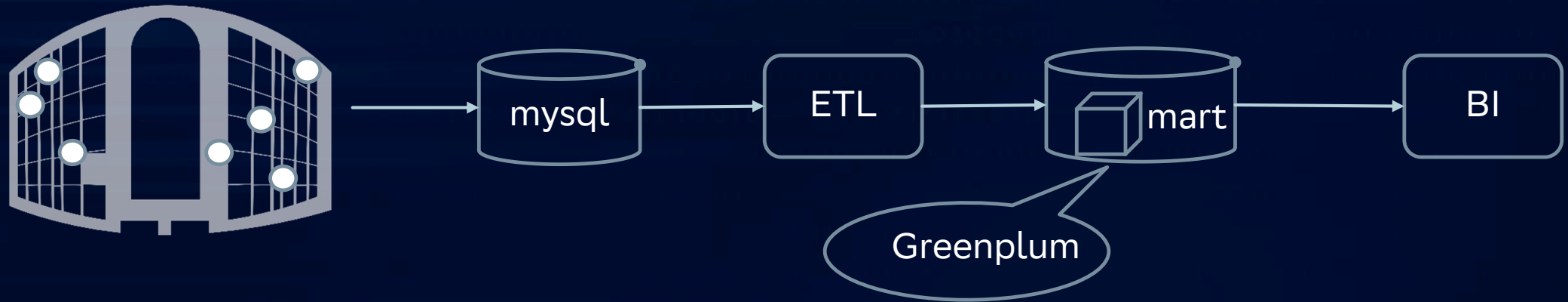


DAG

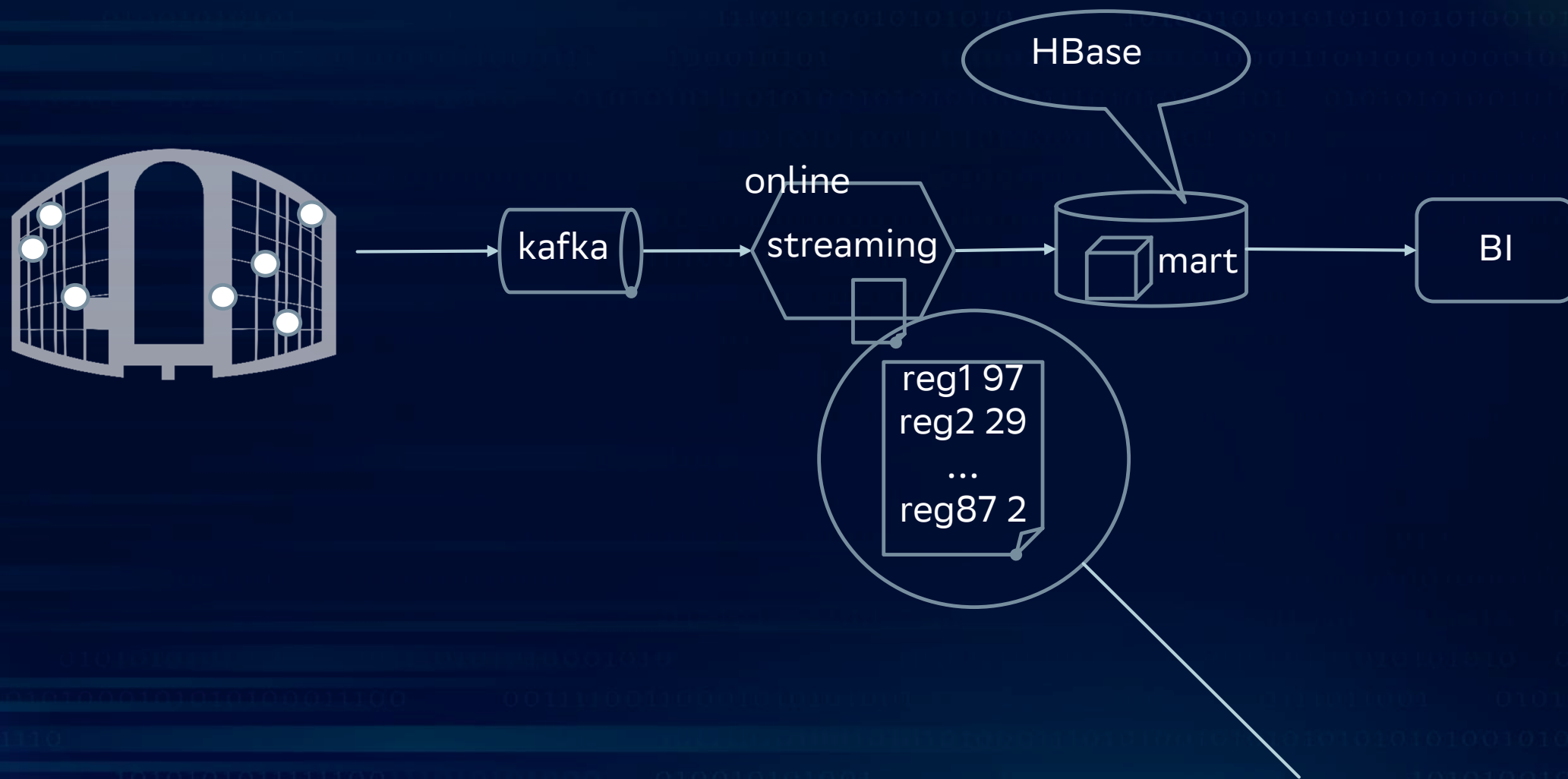
Направленный ациклический граф



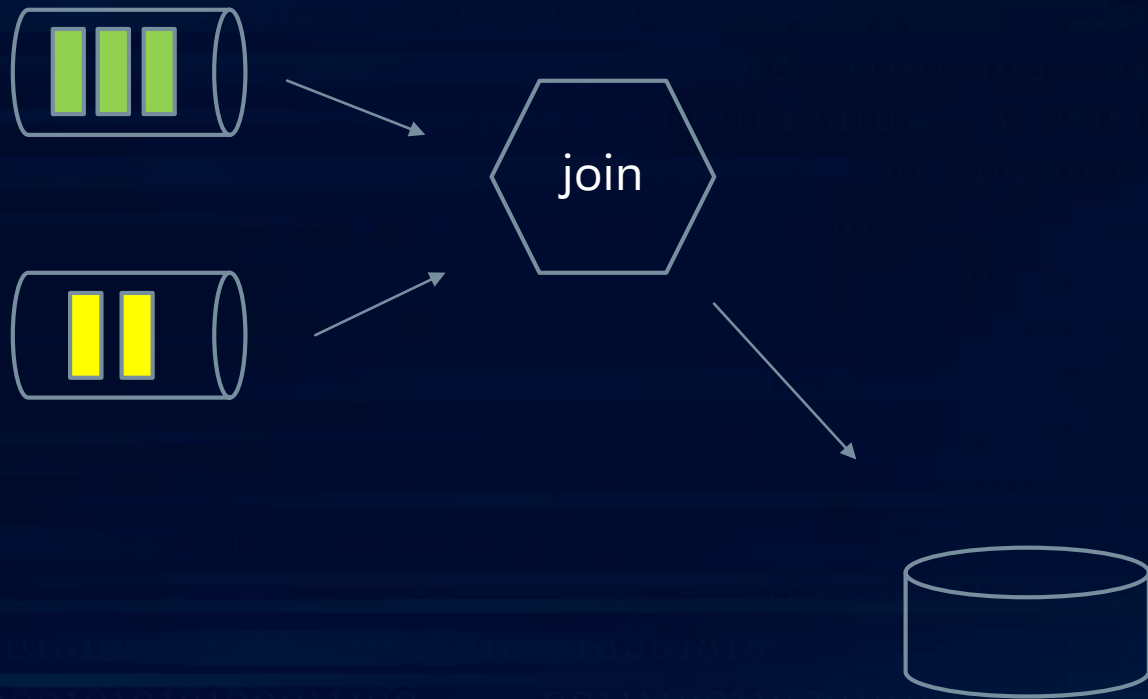
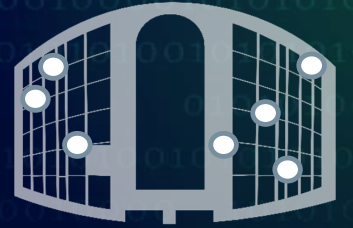
Example 2



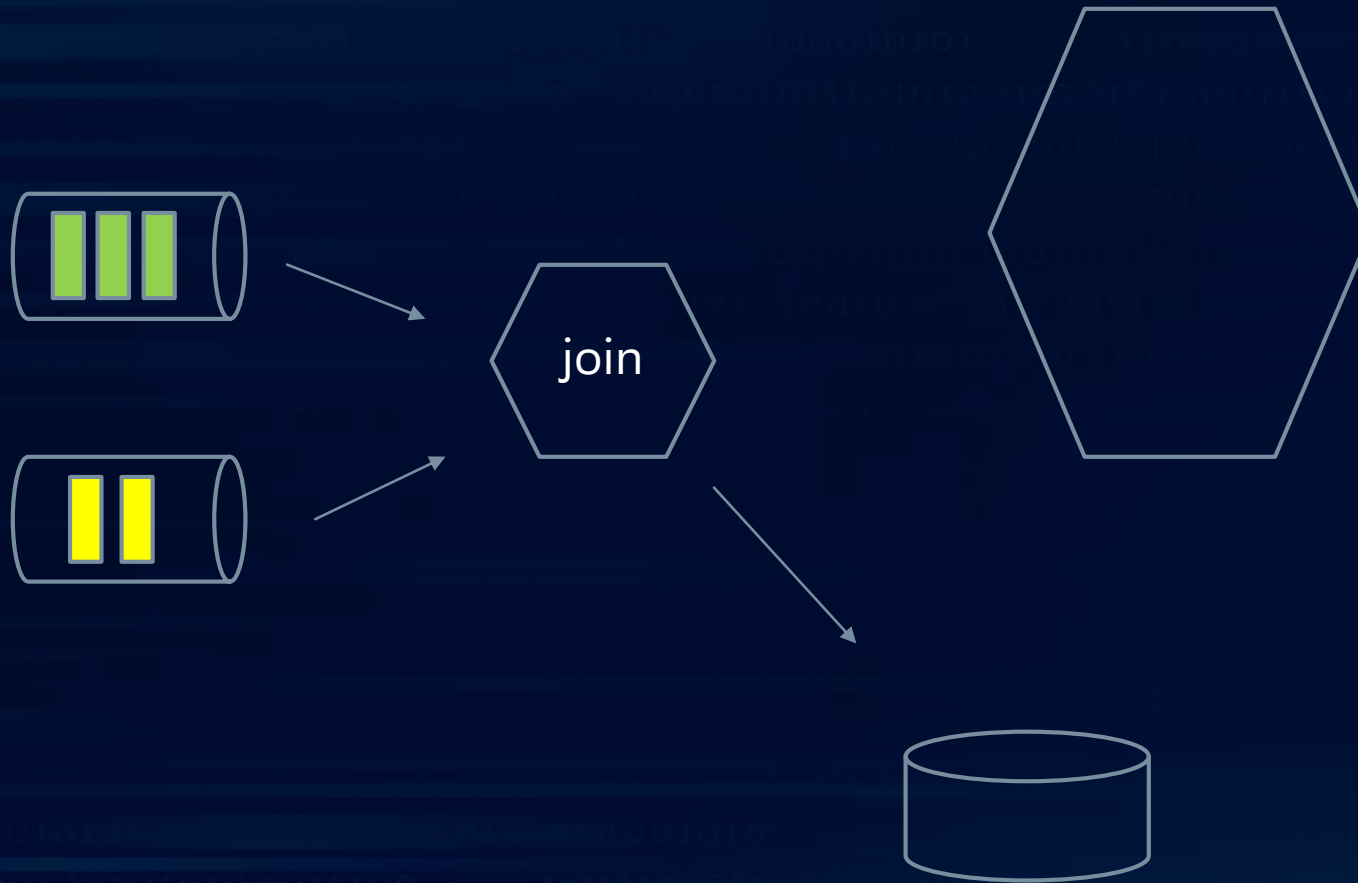
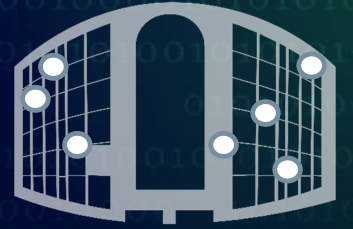
Example 3



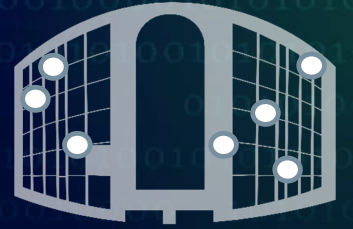
Example 4



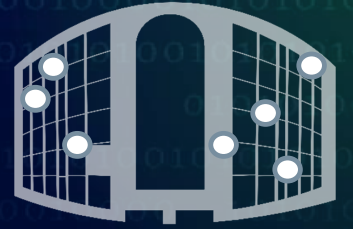
Example 4



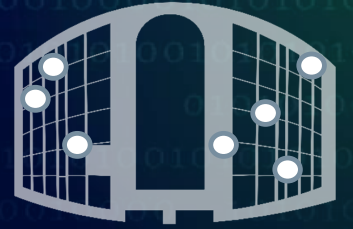
Example 4



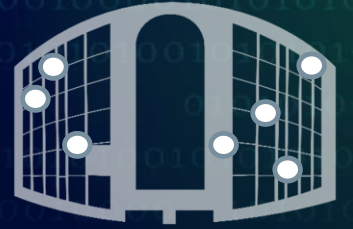
Example 4



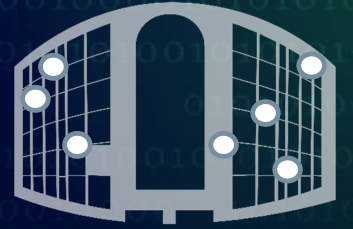
Example 4



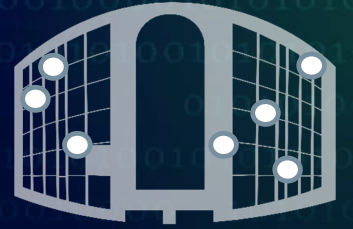
Example 4



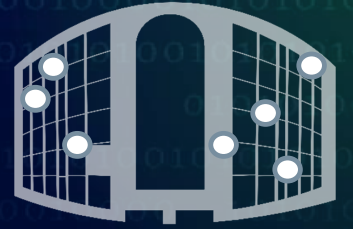
Example 4



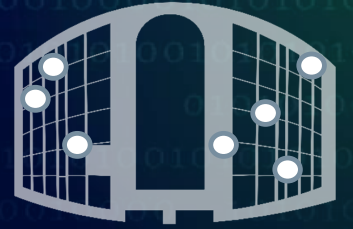
Example 4



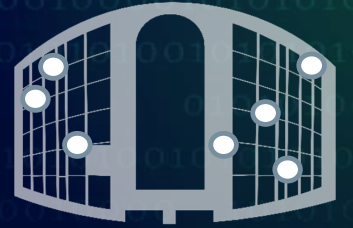
Example 4



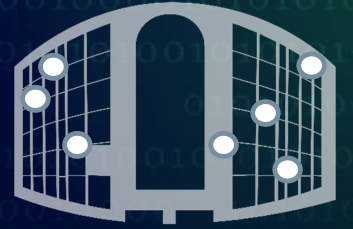
Example 4



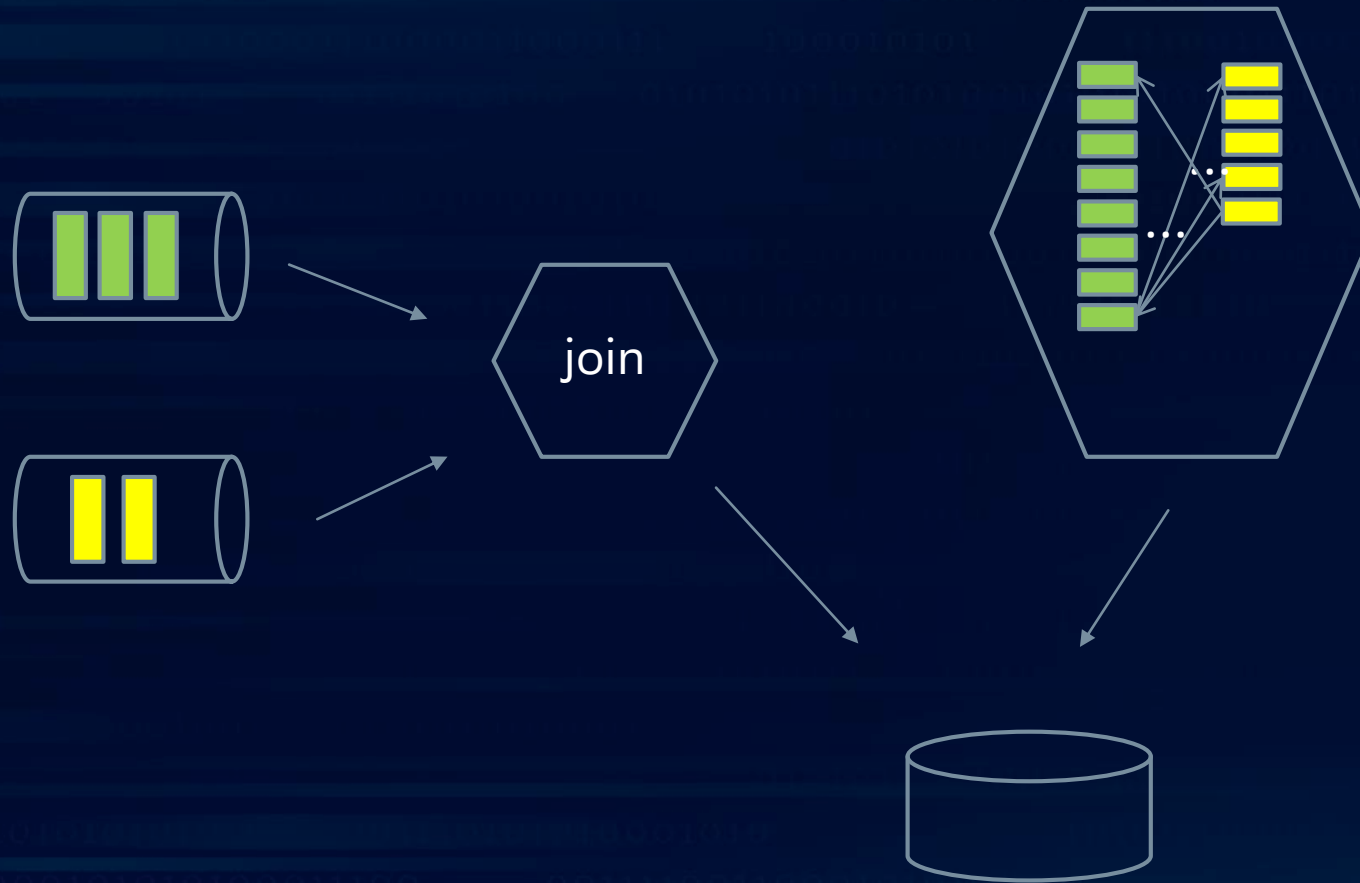
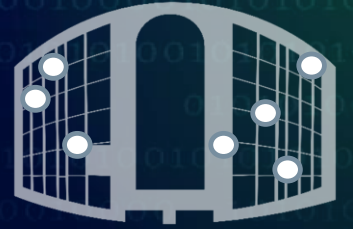
Example 4



Example 4



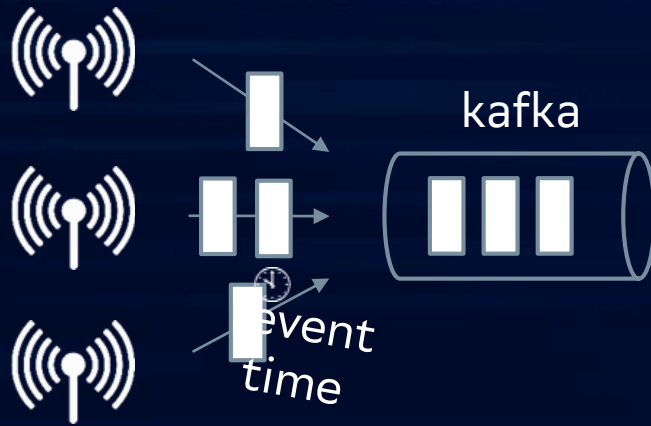
Example 4



Window



Window

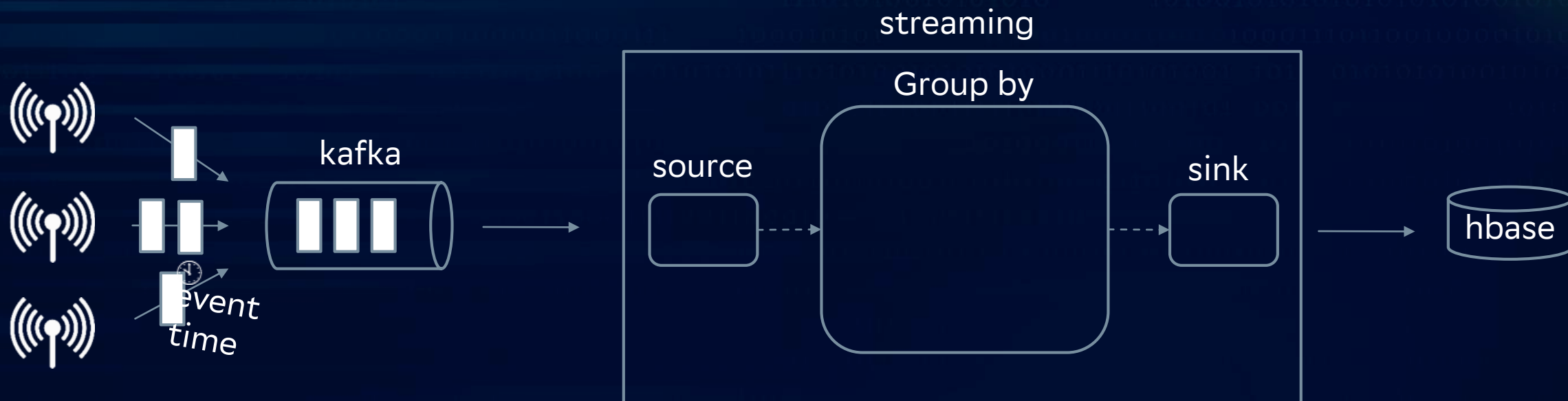


Window

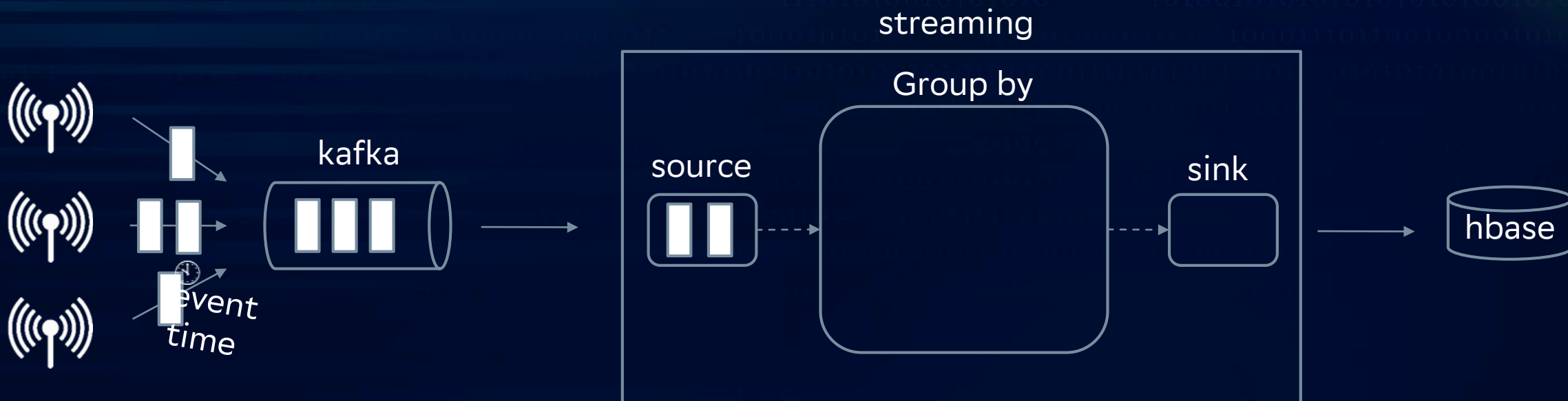
streaming



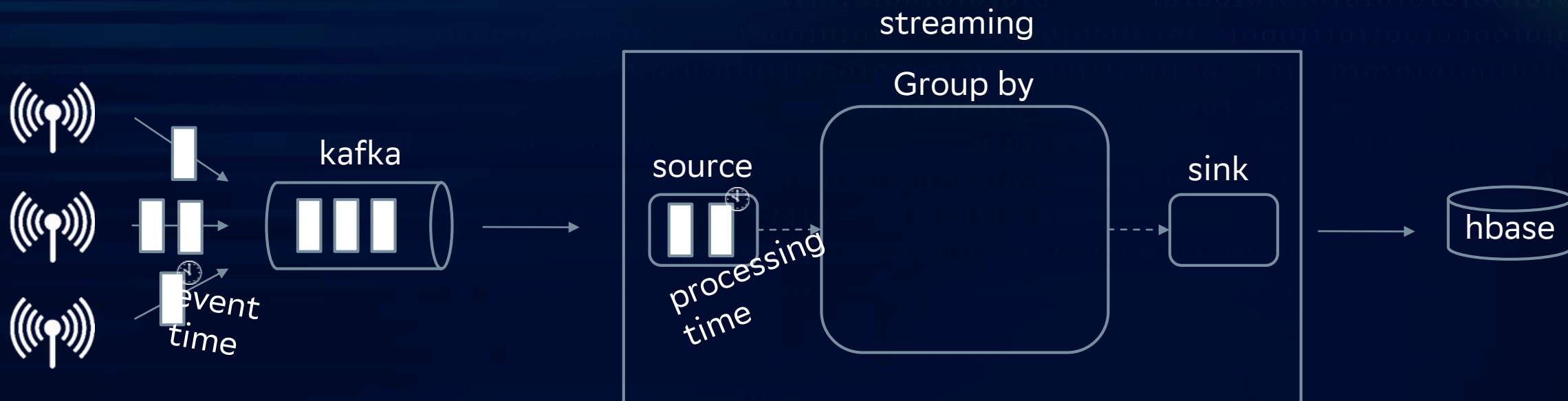
Window



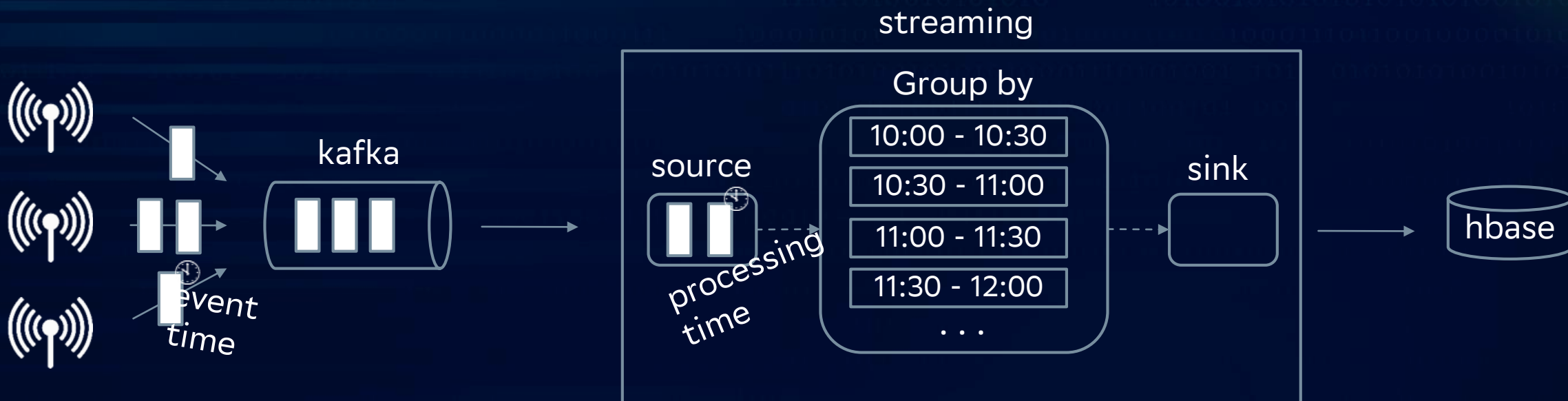
Window



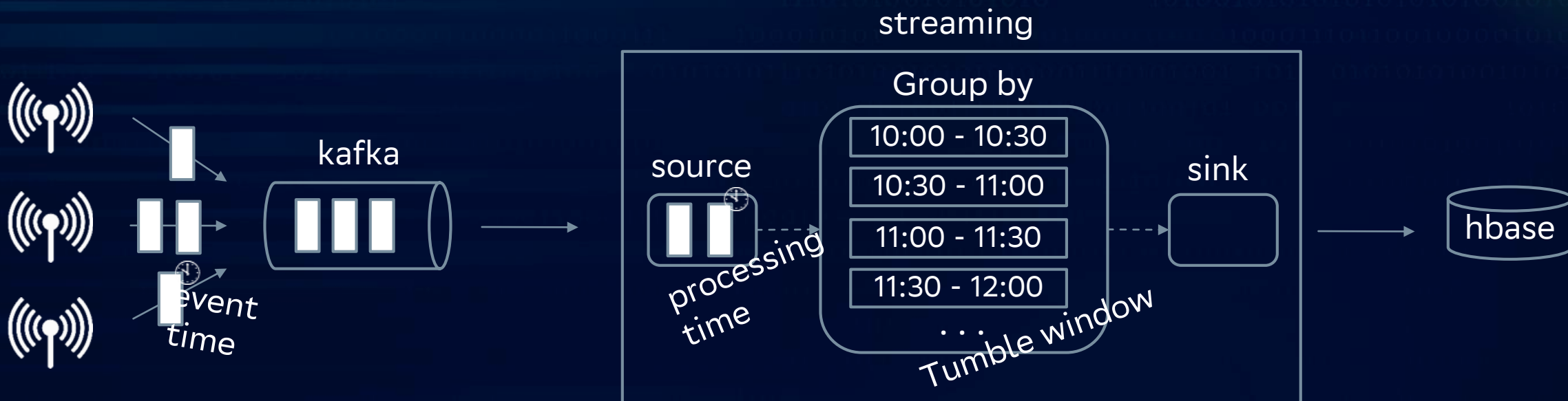
Window



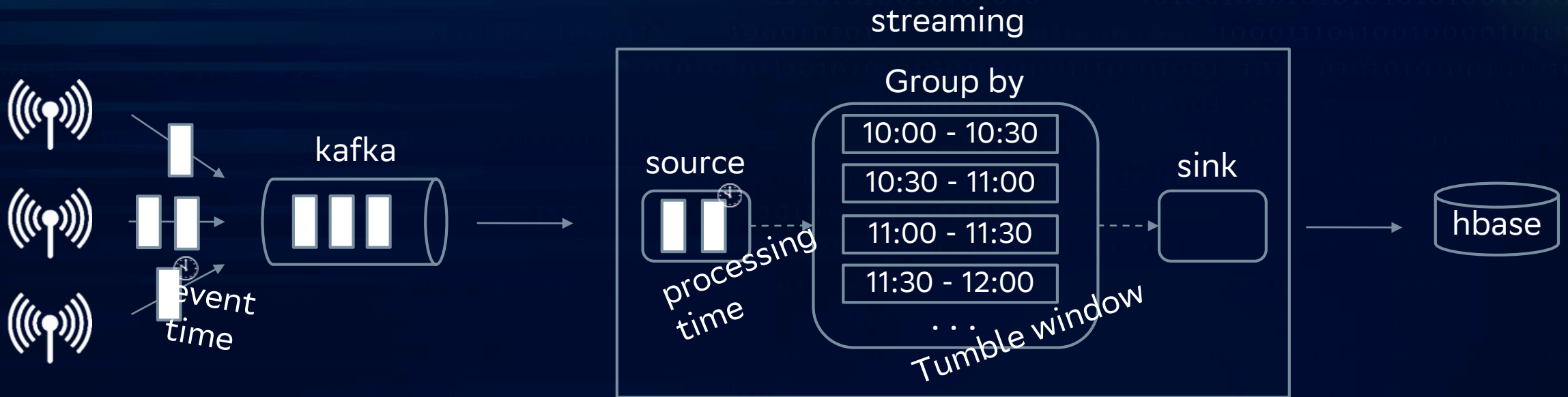
Window



Window



Window



- 👍 Окна бывают разные
- 👍 Сохранение состояния (statefull model)
- 👍 Event time vs processing time
- 👍 Управление через механизм watermarks

Savepoint & checkpoint

Checkpoint – решает задачу обеспечения отказоустойчивости и восстановления после сбоев

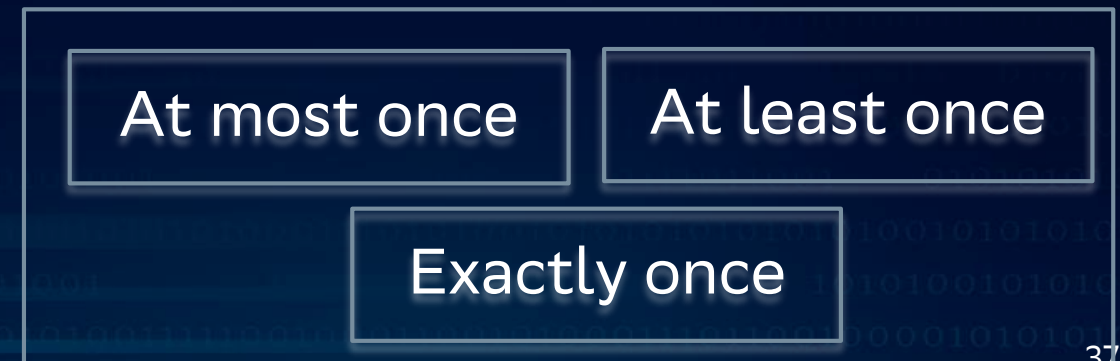
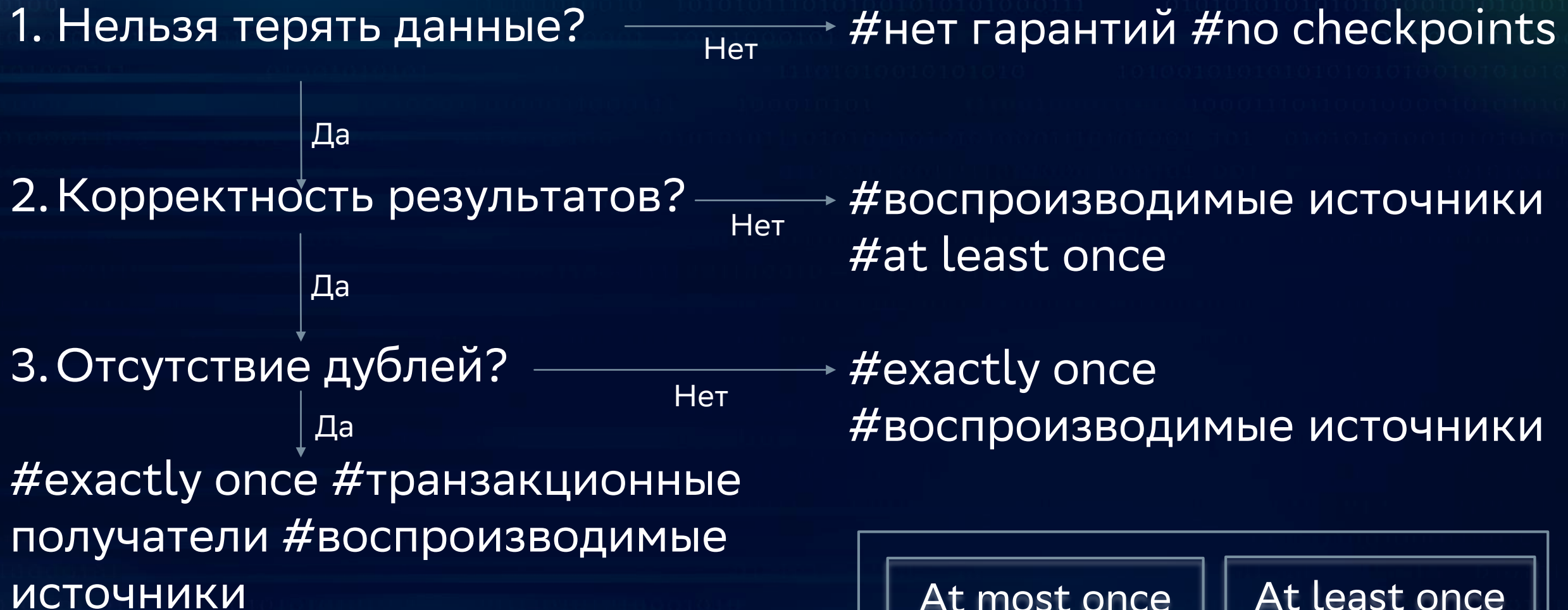
- Регулярность (периодичность?)
- Автоматизация
- Скорость
- Фоновая работа
- Малое потребление ресурсов



Savepoint – механизм ручного сохранения состояния

- Миграция данных/джобов
- Остановка/возобновление
- Новая версия процесса
- Новая версия платформы

Delivery guarantee



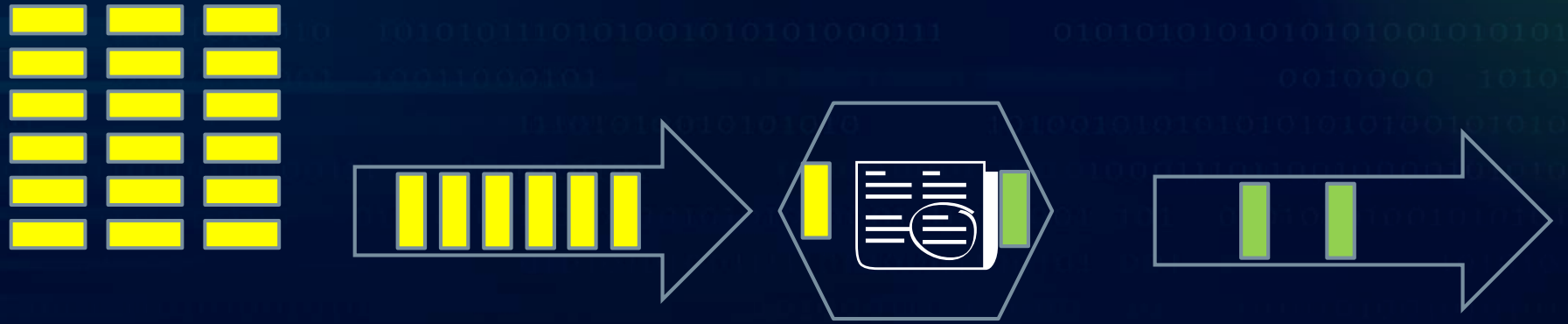
Scaling



Scaling



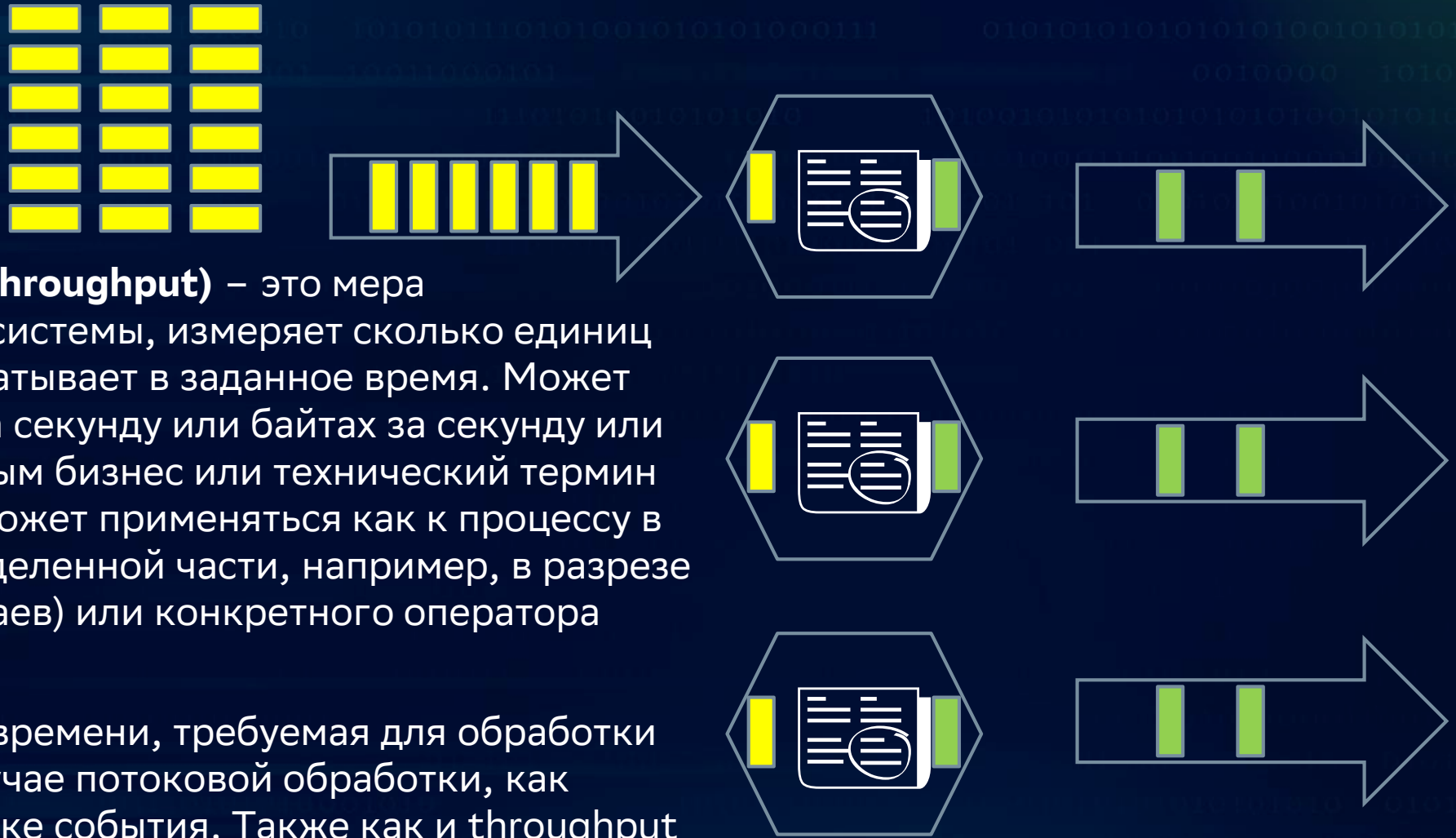
Scaling



Scaling



Scaling



Пропускная способность (throughput) – это мера вычислительной мощности системы, измеряет сколько единиц информации система обрабатывает в заданное время. Может измеряться в сообщениях за секунду или байтах за секунду или другой привязанный к данным бизнес или технический термин за соотв единицу времени. Может применяться как к процессу в целом так и к любой его выделенной части, например, в разрезе задачи (в большинстве случаев) или конкретного оператора данной задачи

Задержка (latency) – мера времени, требуемая для обработки единицы информации. В случае потоковой обработки, как правило говорят об обработке события. Также как и throughput может применяться как к процессу в целом, так и к любой его выделенной части. Как правило речь идет о задаче или об операторе задачи

Data tasks



- Хранение данных, data storage
- Обработка данных, data preparation
- Распространение данных, data acquisition & distribution



- Хранение данных для пакетных задач
- Пакетное распространение данных
- Пакетная обработка данных



- Хранение данных для стриминговых задач
- Потокное распространение данных
- Потокная обработка данных

Environment, data motion

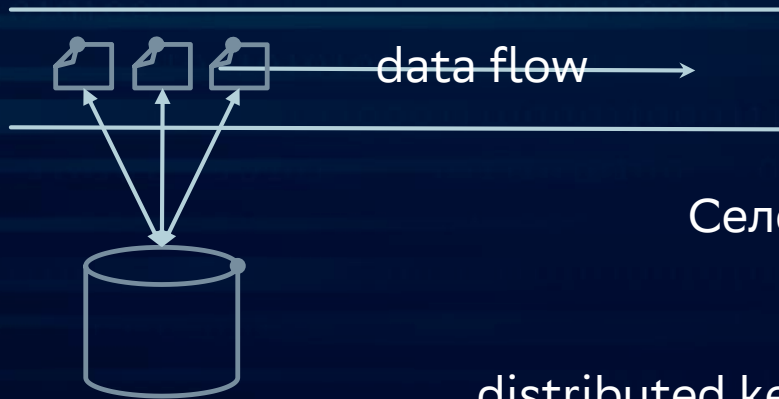


Претенденты на роль трубы: esb, mq, broker, api, ...



world best practice: kafka

Environment, data storage



Селективная работа данных в разрезе одного сообщения

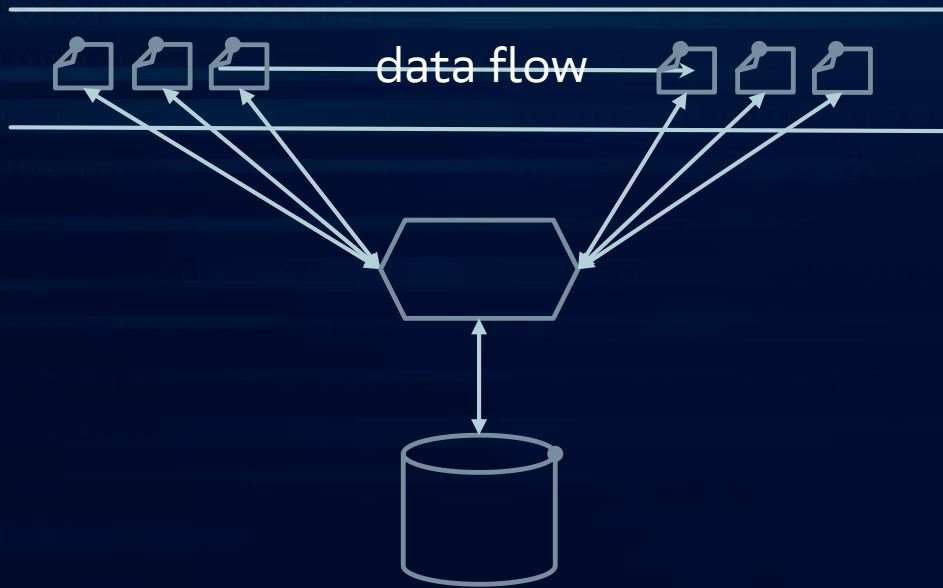
distributed key-value storage: hbase, cassandra, redis, ignite, ...

с учетом экосистемы хадуп



world best practice: compute-manage-storage

Environment, data processing



Популярные фреймворки: spark-streaming, flink, samza, kafka streams, ksqldb, storm, ...

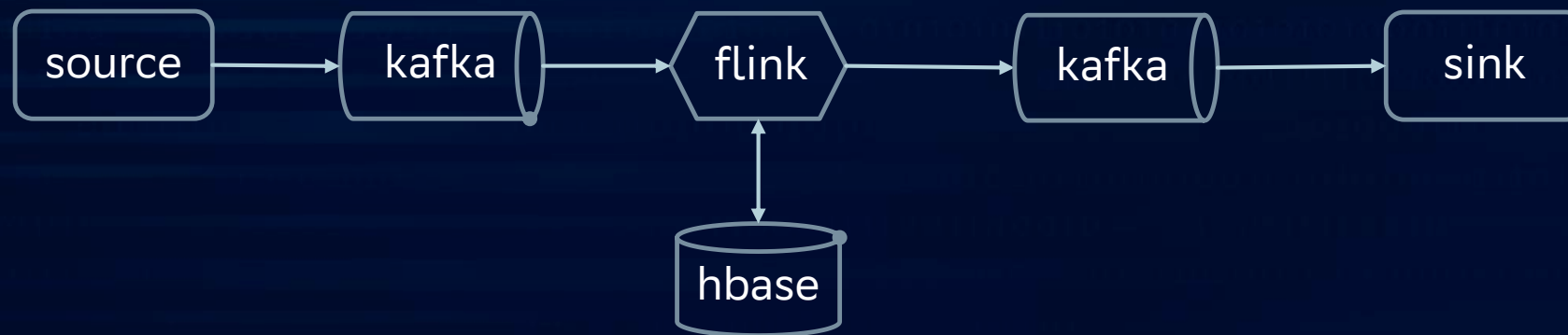
Расскажу и поработаем с flink



Solutions



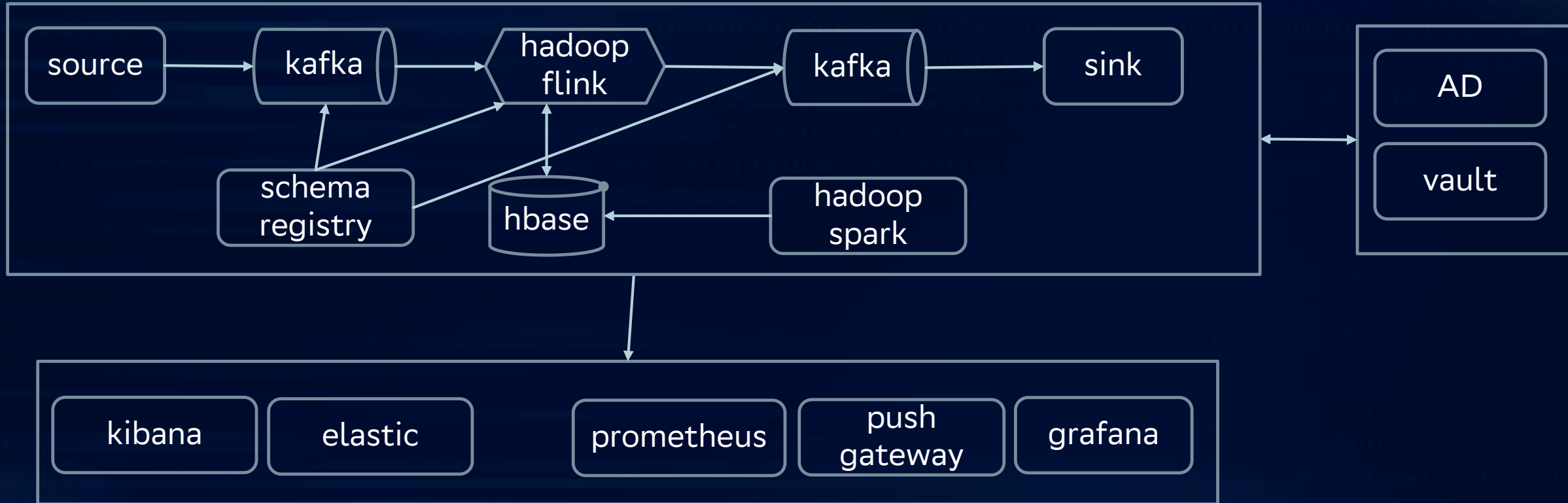
Обогащение данных, онлайн исполнение моделей



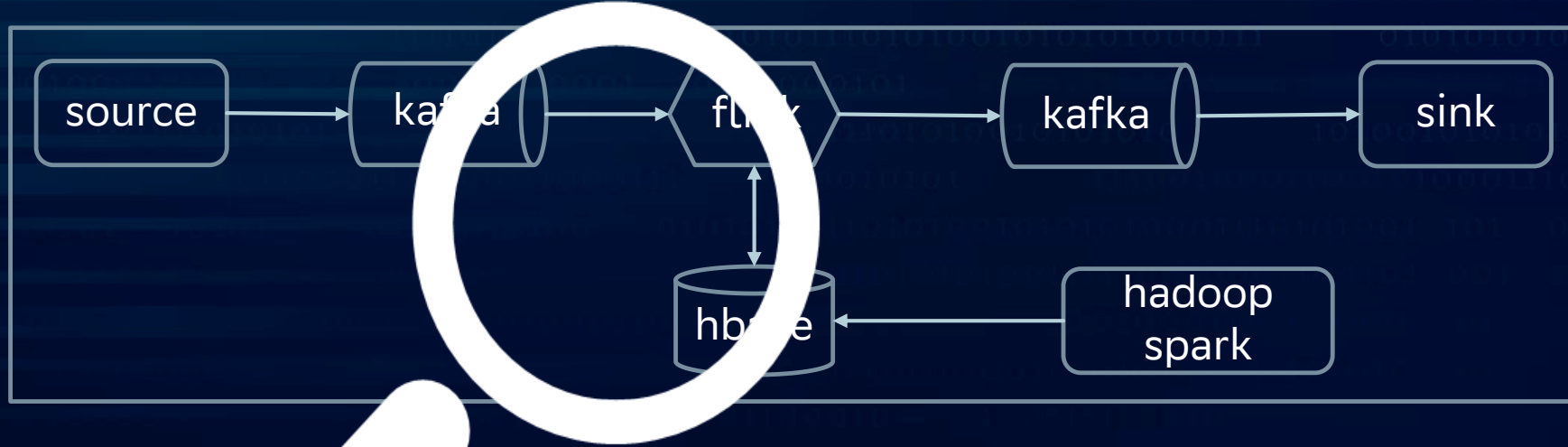
Расчет агрегатов для онлайн аналитики



Environment, hadoop



Deployment



Local

Standalone

YARN

Docker

Kubernetes

flink, overview



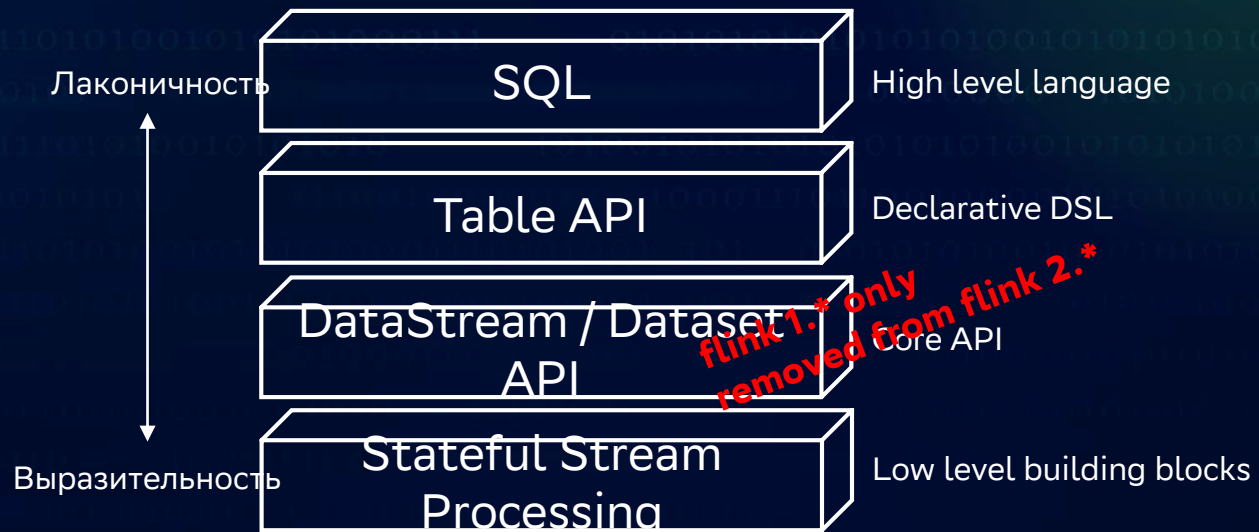
API

```
Table orders = ... //fields: name, total, ...
orders.sqlQuery("select field1, field2 from dataflow
where field3 = 10");
```

```
Table orders = ... //fields: name, total
Table cnt = orders.groupby("name").select("name,
total.count from orders")
```

```
DataStream<Order> ds = ...
ds.keyBy("name").sum("total");
```

```
ds.process(new ProcessFunction<SimpleWords, String>() {
    @Override
    public void processElement(SimpleWords simpleWords, Context context, Collector<String> collector)
    throws Exception {
        collector.collect(simpleWords.word + "; " + simpleWords.cnt);
    }
})
```

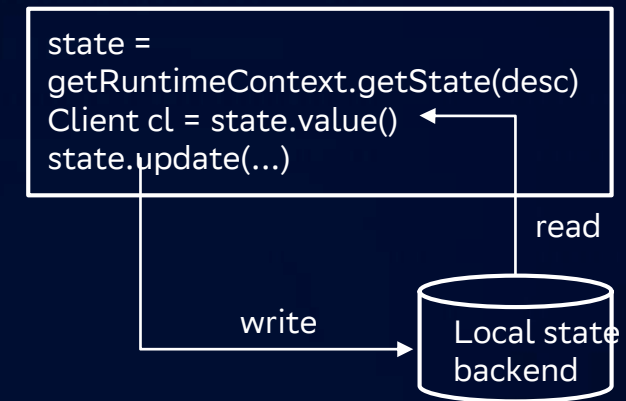


Stateful processing

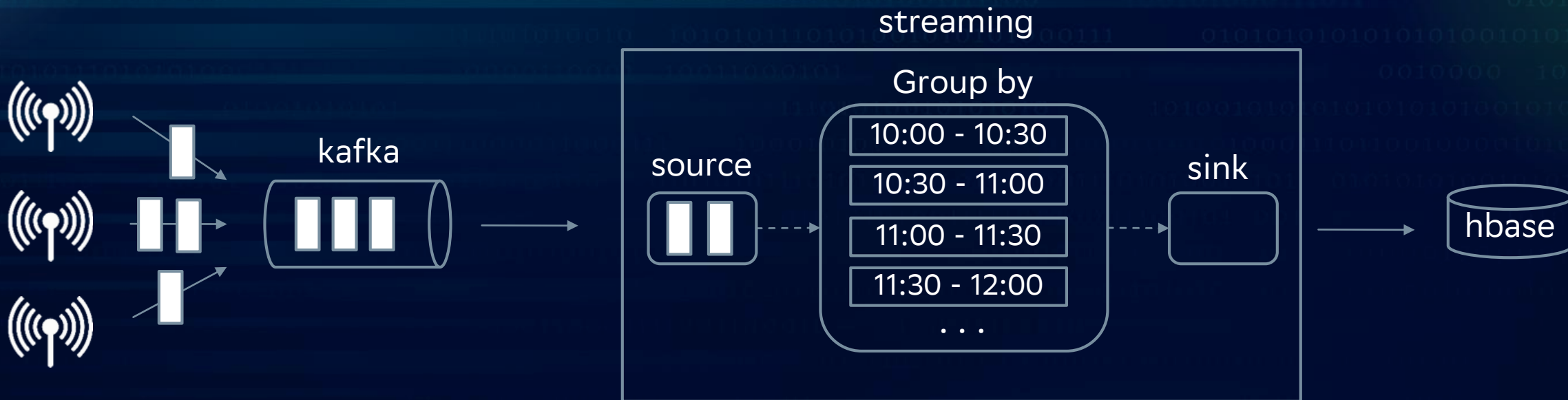
Практически в любом бизнес процессе есть логика по хранению промежуточных результатов и последующего к ним доступа

Управление состоянием

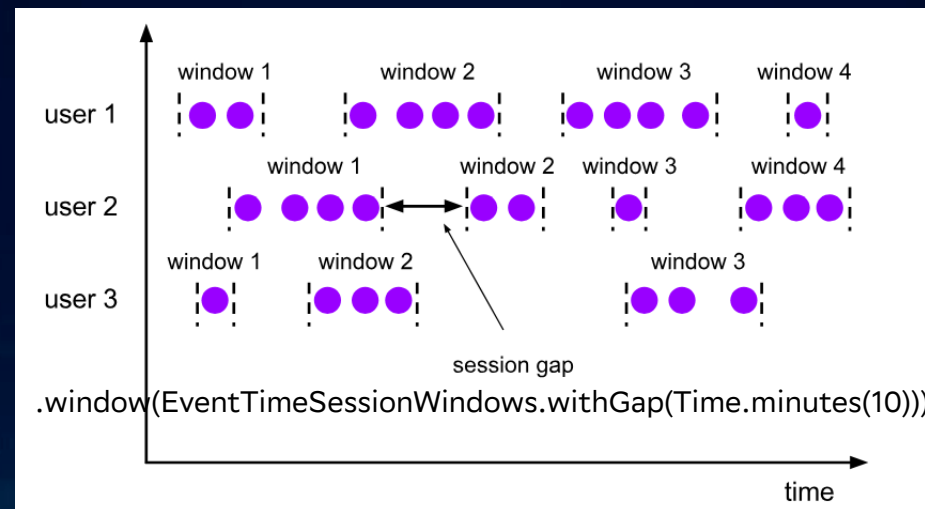
- ✓ Богатый набор примитивов и автоматизация их ведения. Атомарные типы, списки, хештаблицы
- ✓ Настраиваемый backend. in-memory, rocksdb, on-disk data store, custom backend
- ✓ state consistency. Сбои и ошибки прозрачны для разработчика – существующая реализация checkpoint & recovery дает гарантии консистентности состояния и управляемого восстановления
- ✓ Поддержка больших объемов. Благодаря механизмам асинхронного и инкрементального сохранения состояния в нем можно держать терабайты данных.
- ✓ Масштабирование. Перераспределение стейта по узлам вычисления



Работа с событиями



- ✓ Event & processing time mode
- ✓ Window
- ✓ Triggers
- ✓ Watermark support
- ✓ Late data (event) handling



Гарантии доставки

1. Нельзя терять данные?

Нет

#нет гарантий #no checkpoints

Да

2. Корректность результатов?

Нет

#воспроизводимые источники #at least once

Да

3. Отсутствие дублей?

Нет

#exactly once #воспроизводимые источники

Да

#exactly once #транзакционные
получатели #воспроизводимые
источники

At most once

At least once

Exactly once

Инфраструктура и оркестрация

Deployment mode

- ✓ Session. Общий кластер, общие ресурсы, общее управление.
- ✓ Application. Запуск на собственном кластере

Deployment targets

Local

Standalone

YARN

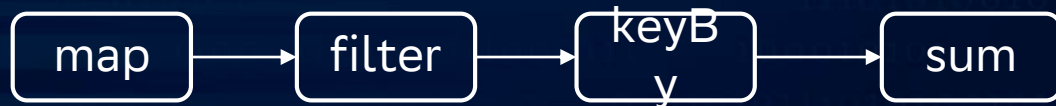
Docker

Kubernetes

Code path

Code `datastream.map().filter().keyBy().sum()`

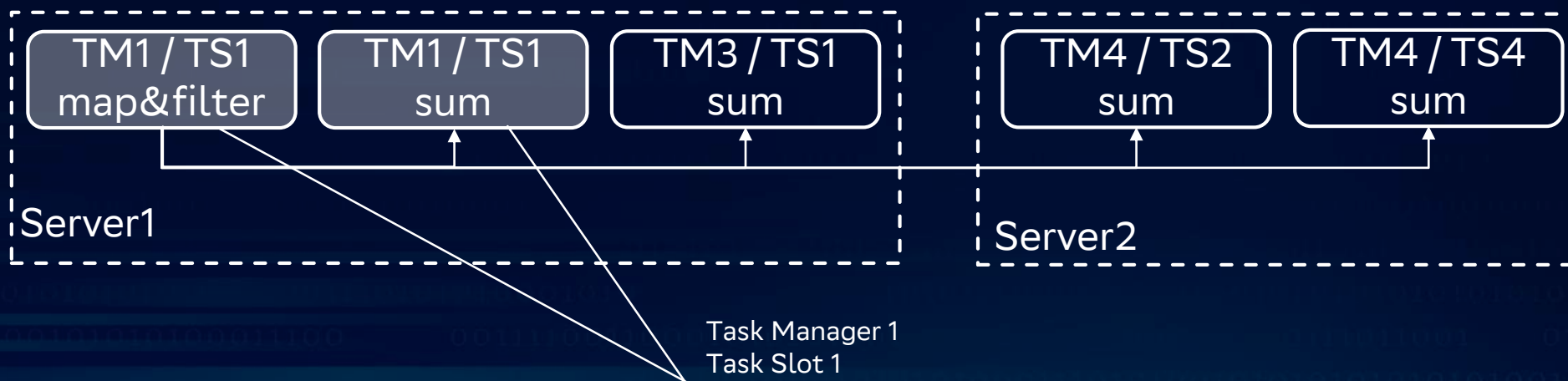
DAG



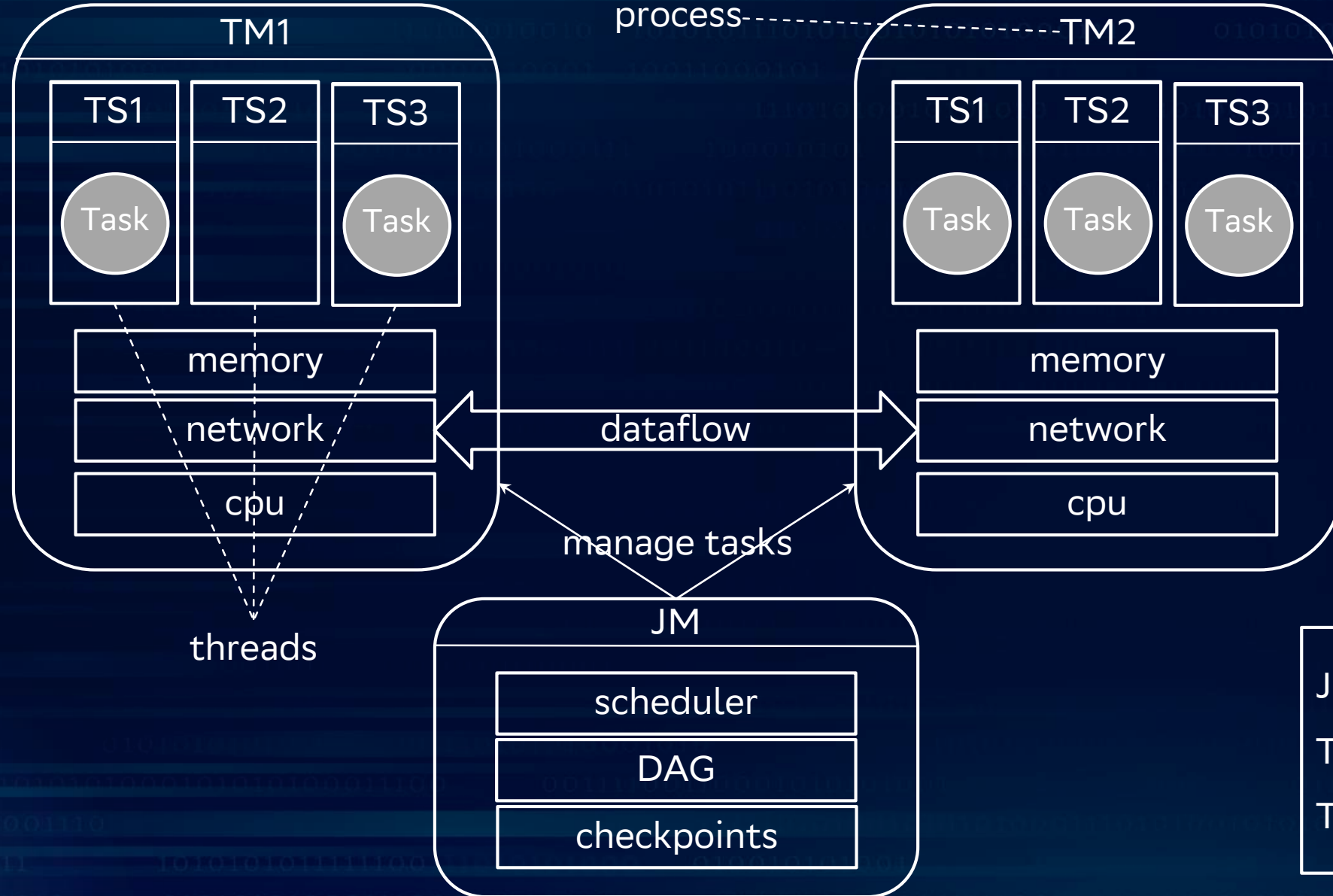
Execution plan



Physical plan



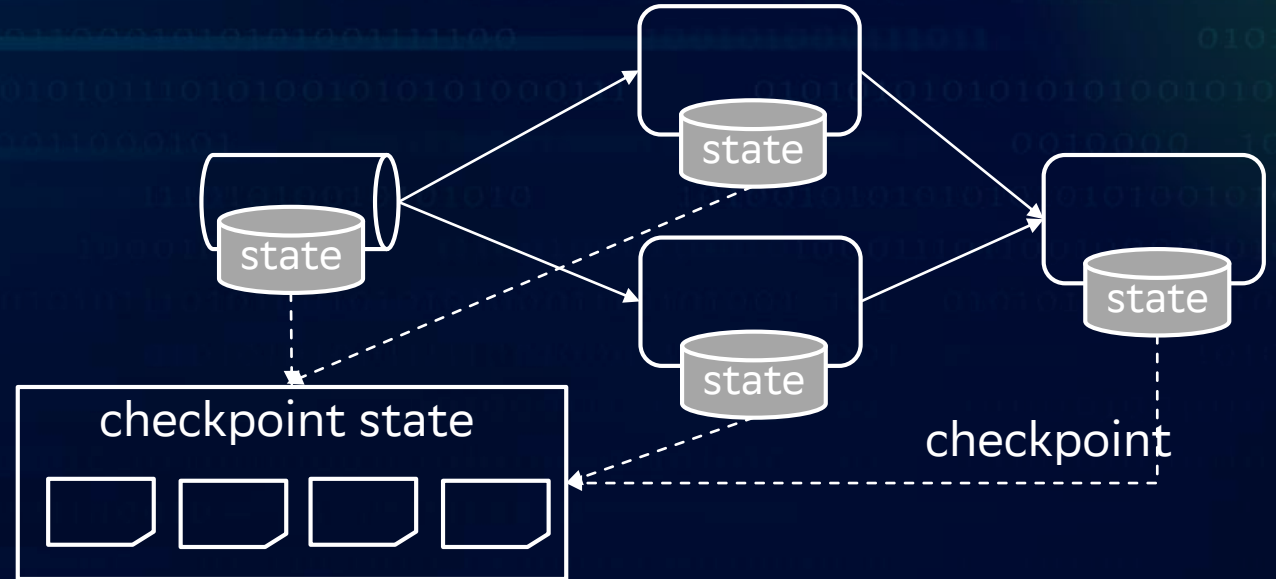
Distribution execution



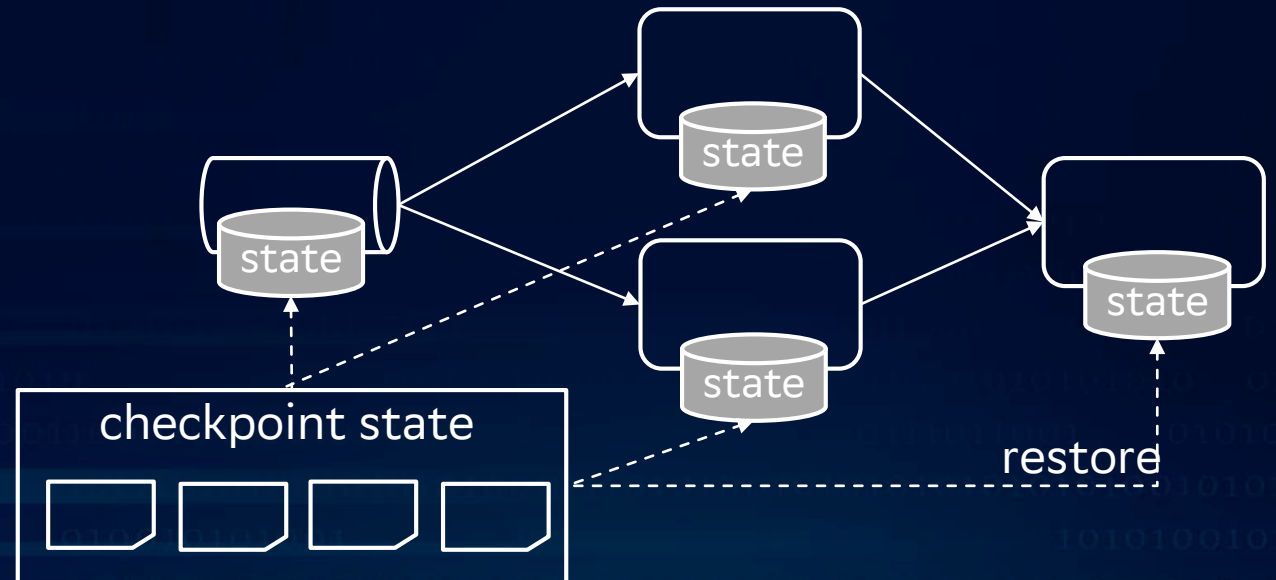
JM = JobManager
TM = TaskManager
TS = TaskSlot

Отказоустойчивость

- ✓ Distributed state snapshots
- ✓ Exactly once guarantee



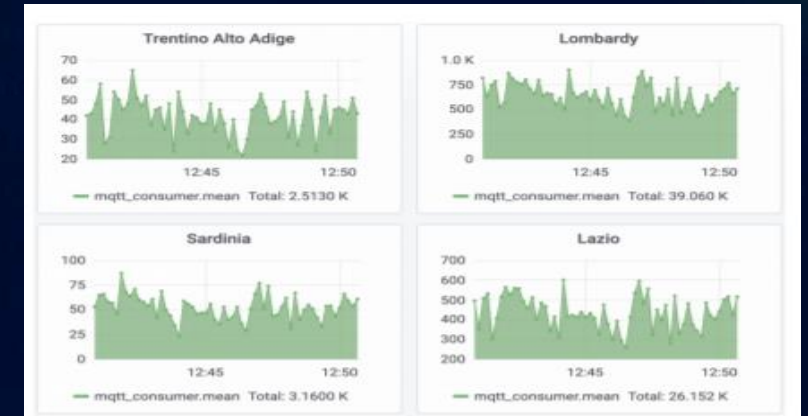
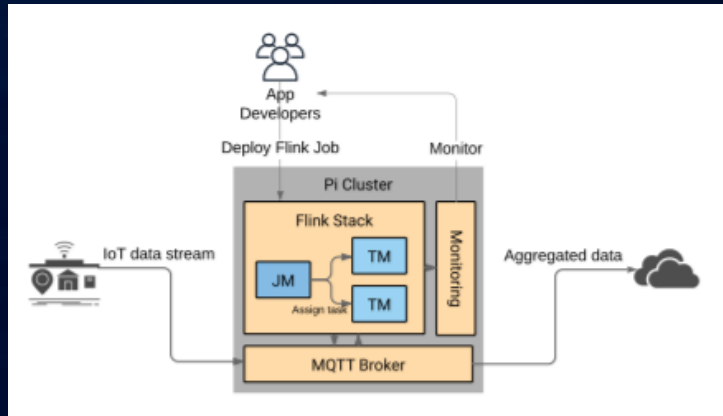
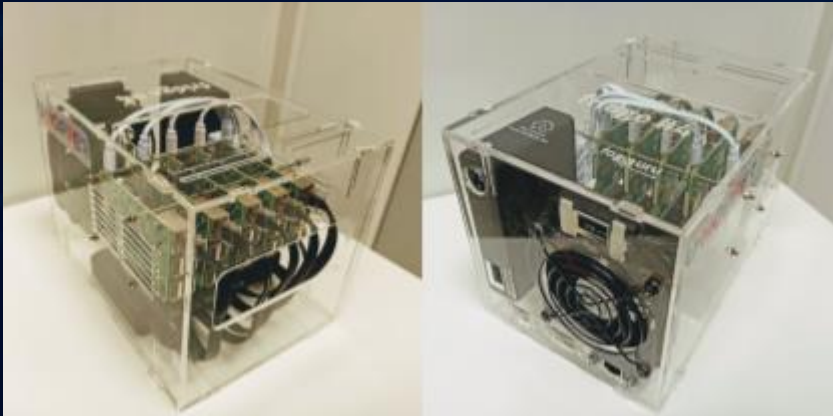
flink 2.0: Disaggregated State Management
state.backend.type: first
table.exec.async-state.enabled: true



Масштабируемость

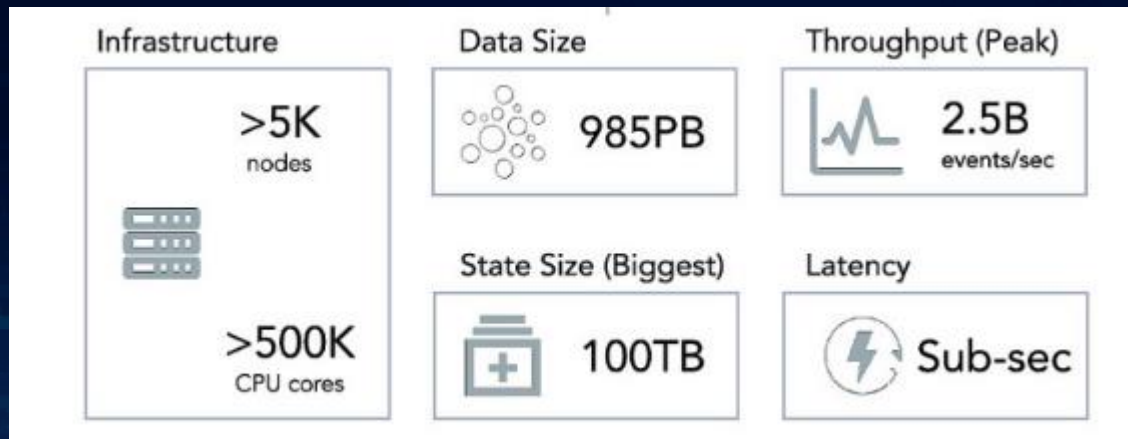
От Raspberry PI

Кластер из 5 Raspberry PI 3B+ Docker Swarm + Flink + Mosquitto Data flow: ~1000 events per sec

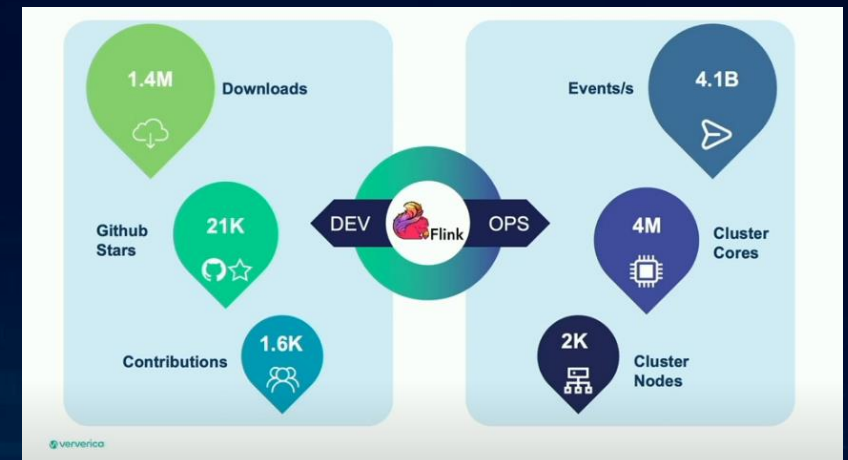


<https://hal.inria.fr/hal-02463206/document>

До Alibaba: double 11 / singles day



<https://sf-2018.flink-forward.org/index.html%3Fp=4068.html>



<https://www.youtube.com/watch?v=YIMyanIBEws>

Throughput & latency

	Record acks (Storm)	Micro-batching (Spark Streaming, Trident)	Transactional updates (Google Cloud Dataflow)	Distributed snapshots (Flink)
Guarantee	At least once	Exactly once	Exactly once	Exactly once
Latency	Very Low	High	Low (delay of transaction)	Very Low
Throughput	Low	High	Medium to High (Depends on throughput of distributed transactional store)	High
Computation model	Streaming	Micro-batch	Streaming	Streaming
Overhead of fault tolerance mechanism	High	Low	Depends on throughput of distributed transactional store	Low
Flow control	Problematic	Problematic	Natural	Natural
Separation of application logic from fault tolerance	Partially (timeouts matter)	No (micro batch size affects semantics)	Yes	Yes

<https://www.ververica.com/blog/high-throughput-low-latency-and-exactly-once-stream-processing-with-apache-flink>

Дополнительно

UI

В разрезе JobManager / оператора. Граф исполнения, статус работы, логи и метрики, состояние снапшотов и периодичность их проведения и т.д.

Metrics

Reporters: JMX, Graphite, InfluxDB, Prometheus, PrometheusPushGateway, StatsD, Datadog, Slf4j

Type: CPU, Memory, Threads, GarbageCollection, ClassLoader, Network, Cluster, Availability, Checkpointing, RocksDB, IO, Connectors, System resources

Scope: User Scope, System Scope, List of all Variables, User Variables

Fine tuning

Сотни (~500) конфигурационных параметров

REST-API

Управление задачами посредством вызовов rest-api

Queryable state

Доступ к данным потока исполнения через sql

Дополнительно

Backpressure, Checkpoint progress, recovery state, etc

Дополнительно

Stateful Computations over Data Streams

Apache Flink is a framework and distributed processing engine for stateful computations over unbounded and bounded data streams. Flink has been designed to run in all common cluster environments, perform computations at in-memory speed and at any scale.



- ✓ State management is what makes flink powerful
- ✓ Consistent, one-at-a-time event processing is what makes flink flexible

Community



<https://flink.apache.org/>



<https://flink-packages.org/>



<https://www.ververica.com/>



<https://eu.alibabacloud.com/>



<https://www.flink-forward.org/>



<https://cwiki.apache.org/confluence/display/FLINK/Apache+Flink+Home>



<https://github.com/apache/flink>

Homework



Термины, которые полезно знать

- *broker*
- *key-value storage*
- *state, stateless vs statefull*
- *dag*
- *async*
- *latency & throughput*
- *поток*
- *гарантии доставки*
- *стриминг*
- *ВИДЫ ОКОН*

Questions



Спасибо за внимание

Homework



Streamhouse

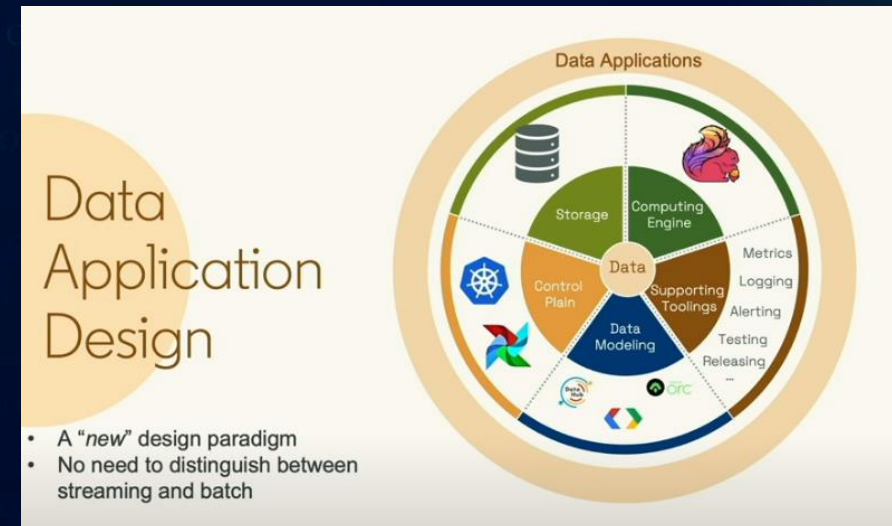
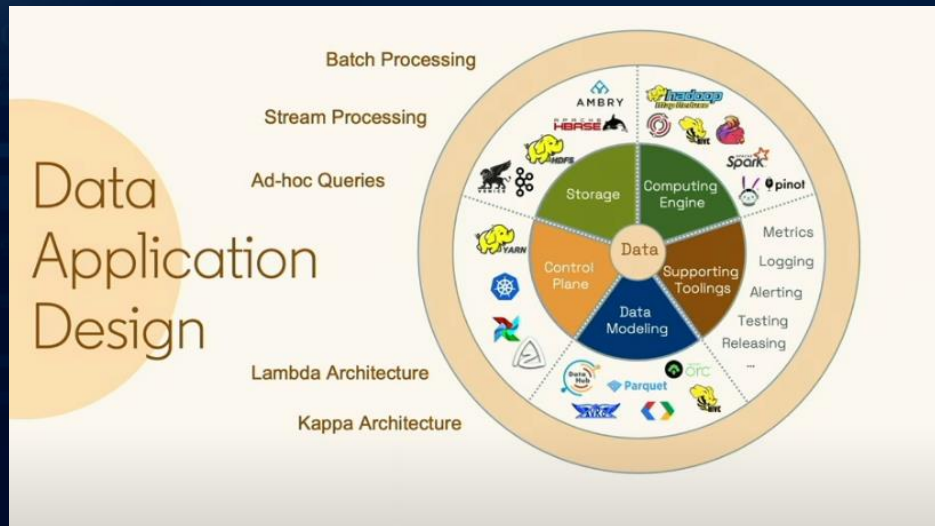


<https://www.ververica.com/streamhouse>

Термины, которые полезно знать

- *broker*
- *key-value storage*
- *state, stateless vs statefull*
- *dag*
- *async*
- *latency & throughput*
- ПОТОК
- ГАРАНТИИ ДОСТАВКИ
- СТРИМИНГ
- ВИДЫ ОКОН

LinkedIn's story

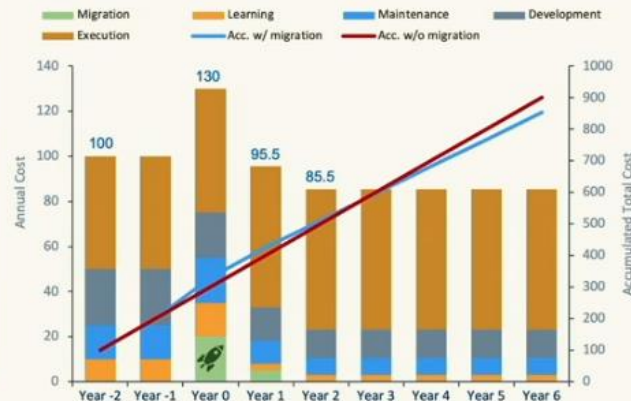


A Toy Example

Cost Estimations

Cost Category	Before	After
Migration	0	30 (30% of Original Annual Spending)
Learning	10	3 (-70%)
Maintenance	15	7.5 (-50%)
Development	25	12.5 (-50%)
Execution	50	62.5 (+25%)
Total	100	85.5 (-14.5%)

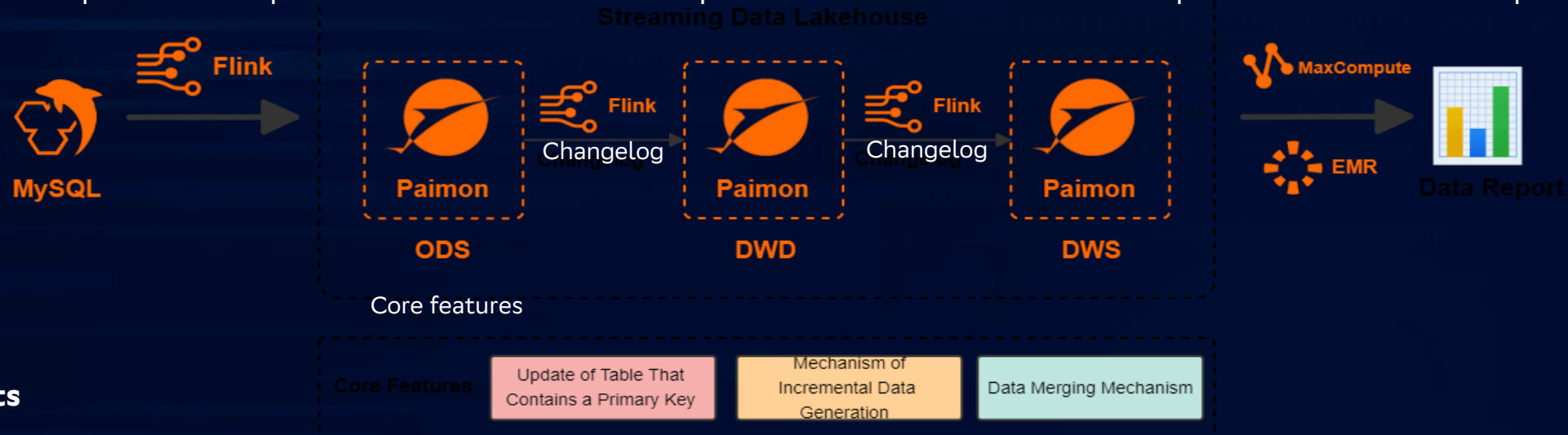
Data Infra Cost Over Years



<https://www.youtube.com/watch?v=ImXFxB5hXs0>

Alibaba cloud

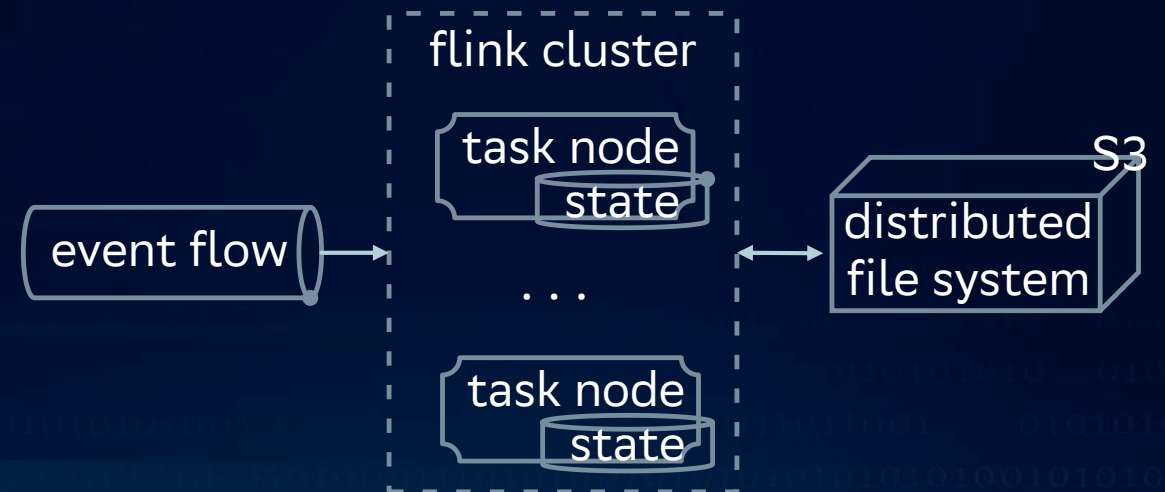
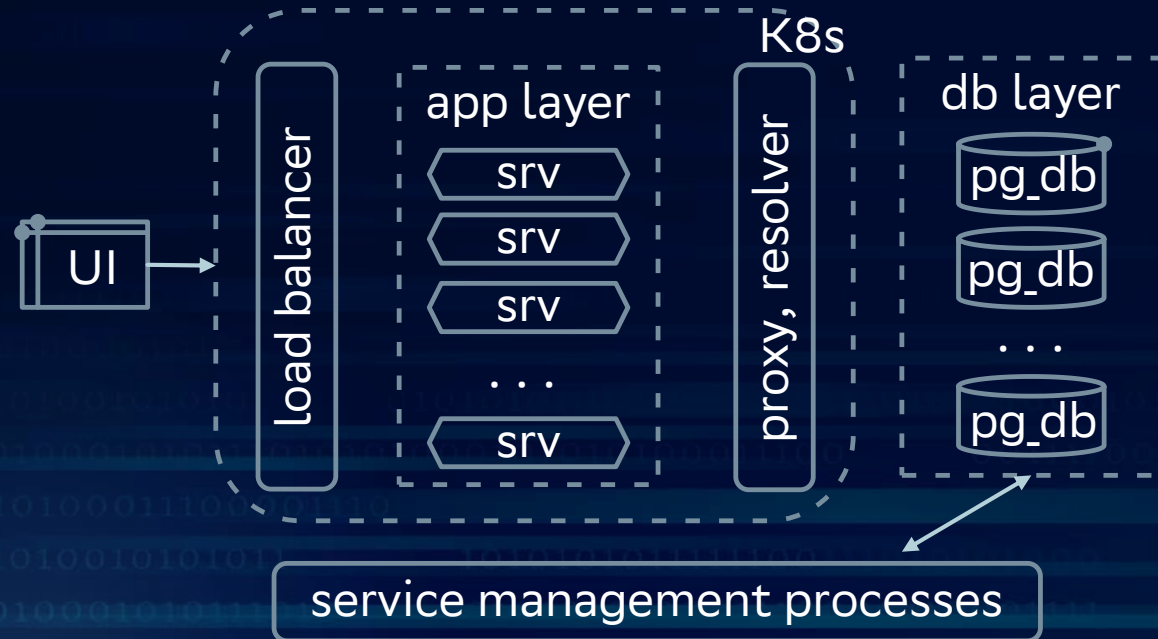
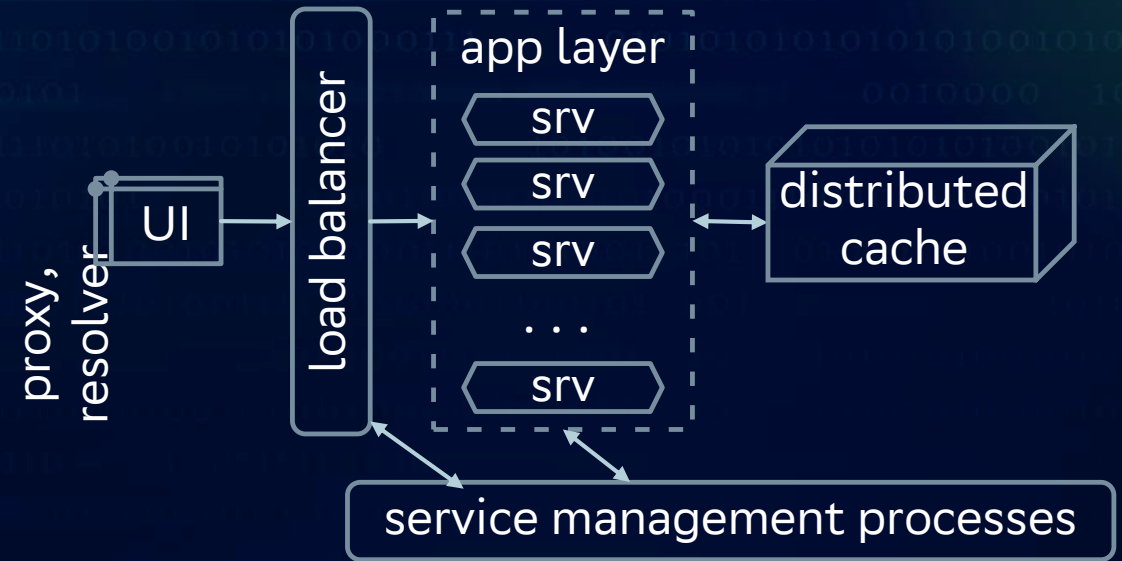
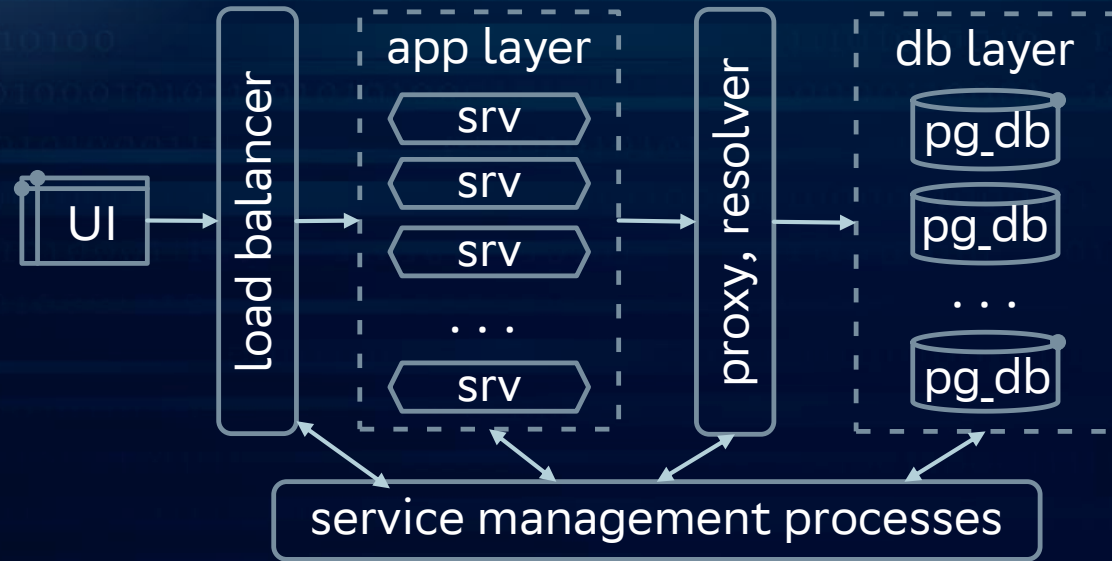
1. Realtime Compute for Apache Flink writes data from a data source to Apache Paimon to form the ODS layer.
2. Realtime Compute for Apache Flink subscribes to changelogs at the ODS layer for processing and then rewrites the data to Apache Paimon to form the DWD layer.
3. Realtime Compute for Apache Flink subscribes to changelogs at the DWD layer for processing and then rewrites the data to Apache Paimon to form the DWS layer.
4. MaxCompute or E-MapReduce reads data from an Apache Paimon external table and provides external data queries.



Benefits

- Each layer of Apache Paimon can deliver changelogs to the downstream storage within a minute-level latency. This reduces the latency of traditional offline data warehouses from hours or even days to minutes.
- Each layer of Apache Paimon can directly receive changelogs. Data in partitions is not overwritten. This greatly reduces the cost of updating and revising data in traditional offline data warehouses and resolves the issues that data at the intermediate layer is not easy to query, update, or correct.
- This solution provides a unified model and uses a simplified architecture. The logic of extract, transform, and load (ETL) operations is implemented based on Flink SQL. All data at the ODS layer, DWD layer, and DWS layer is stored in Apache Paimon. This reduces the architecture complexity and improves data processing efficiency.

architecture shards vs cache vs flink-streaming



compare shards vs cache vs flink-streaming

	schards	cache	flink
Обеспечивающий слой	Полноценная разработка и сопровождение	Либо собственная разработка либо реализация средствами платформы оркестрации	Доступно из коробки
Масштабирование	Ручник В Сбере: 100+ серверов, ~100k tps В мире: ?	Полуручное В Сбере: 100+ серверов, ~100k rps В мире: ?	Автоматизированное В Сбере: 100+ серверов, ~100k rps В мире: 1000+ серверов, ~1B rps
Путь java-разработчика	java, hibernate	java, no-sql api	java, sql
Применение	Возможна любая сложная логика	Только простые операции вида put-get	Не сложные sql запросы, не блокирующие java-задачи
Обработка данных	+OLTP +OLAP +DataLake +Streaming	-OLTP -OLAP -DataLake +Streaming	+OLTP +OLAP +DataLake +Streaming